MACHINE LEARNING

Learning a Model: Decision Trees

Chapter 2

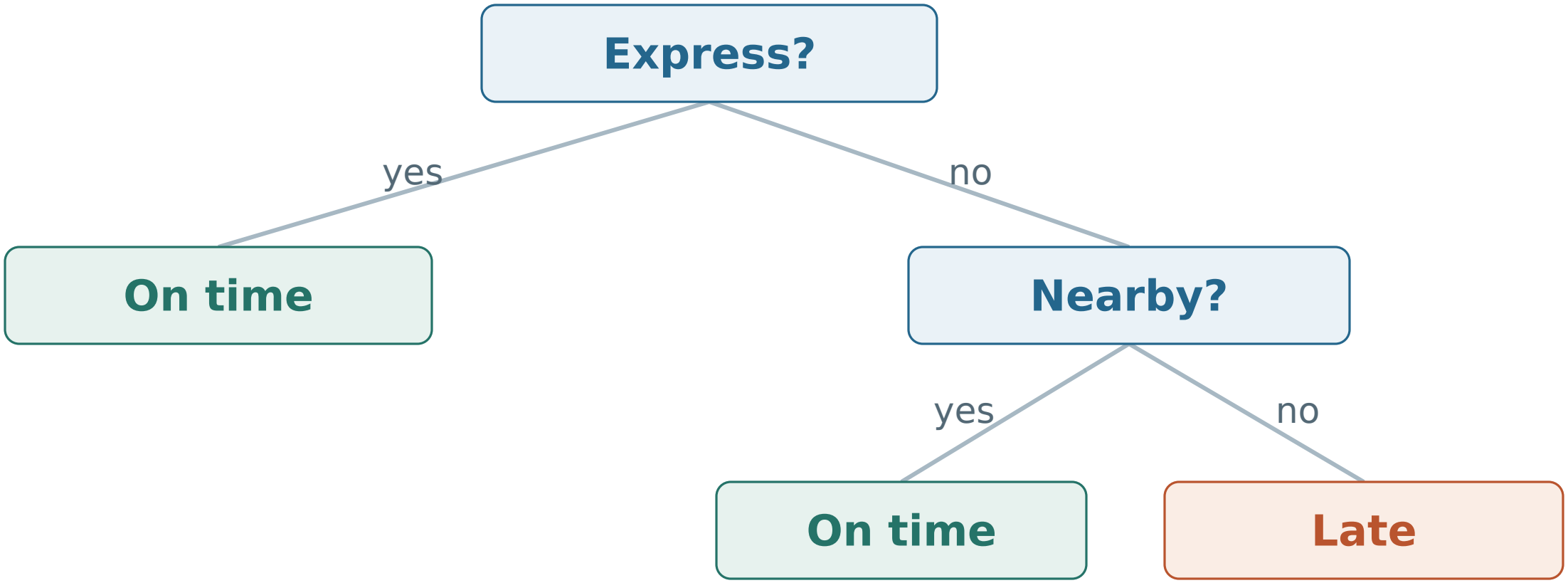
The choice of model family

Family	The set of prediction rules the learner is allowed to consider.
Fitting	Use the training samples to choose one particular rule in that set.
This chapter	Replace the nearest-neighbor form of prediction with a branching rule.

Trees and nearest neighbors

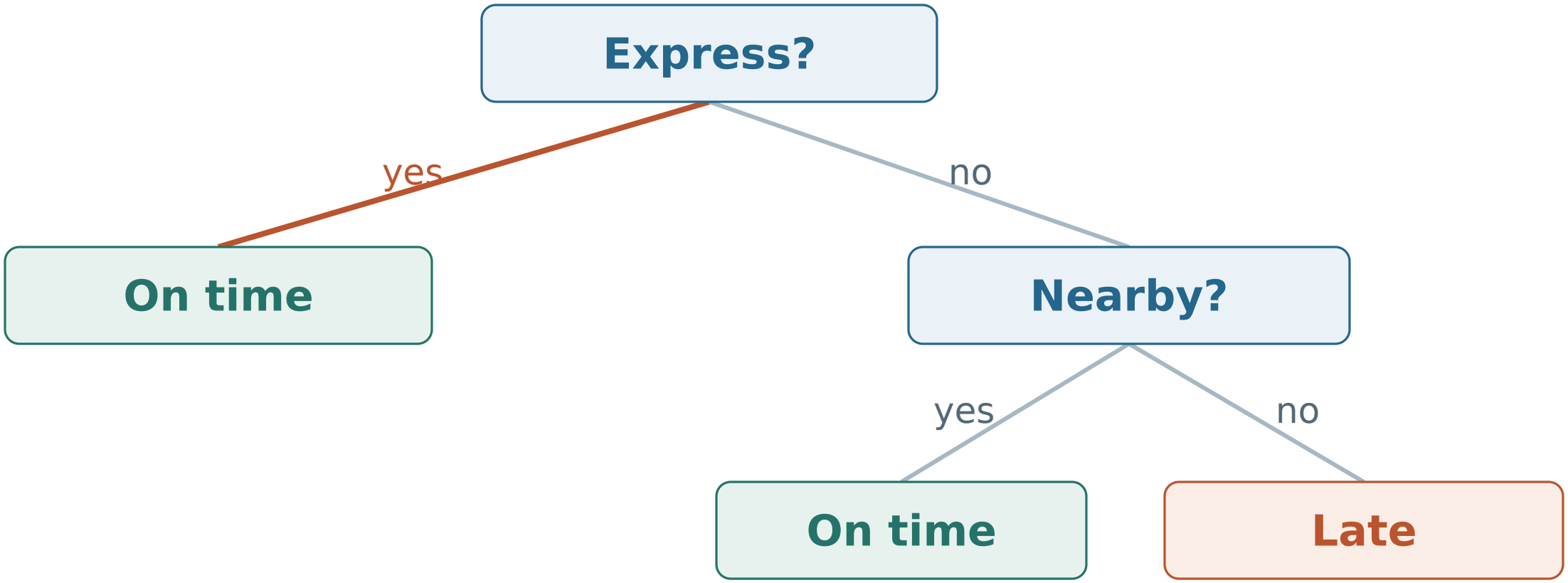
Nearest neighbors	Compare the new input with retained labeled samples.
Decision tree	Follow learned feature questions to a stored leaf prediction.
Where the work occurs	Tree fitting searches for the questions. Prediction applies the resulting rule.

A tree for delivery prediction



Root: first question. Internal nodes: tests. Branches: answers. Leaves: predictions.

Prediction follows one path



New input: express=yes, nearby=no. Prediction: on time.

The first case: classification with nominal features

Target	A class label, such as on time / late or spam / not spam.
Features	Nominal categories. Our opening examples use yes / no inputs.
Questions	A feature category or group of categories separates the samples.
Later	Numerical inputs use thresholds. Numerical targets lead to regression trees.

Four delivery training samples

Delivery	Express?	Nearby?	Observed outcome
A	yes	yes	on time
B	yes	no	on time
C	no	yes	on time
D	no	no	late

The two features are inputs. The observed outcome is the training target.

A one-leaf tree

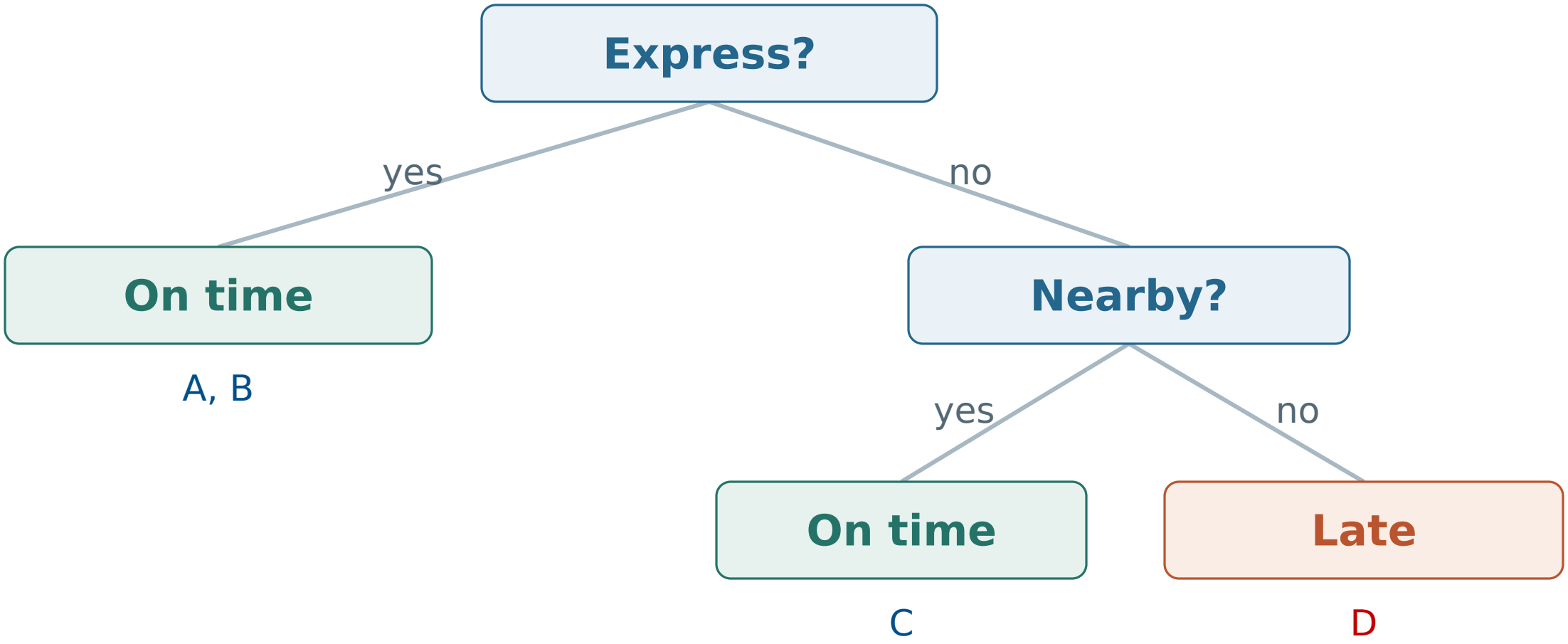


1 leaf

3 correct, 1 mistake

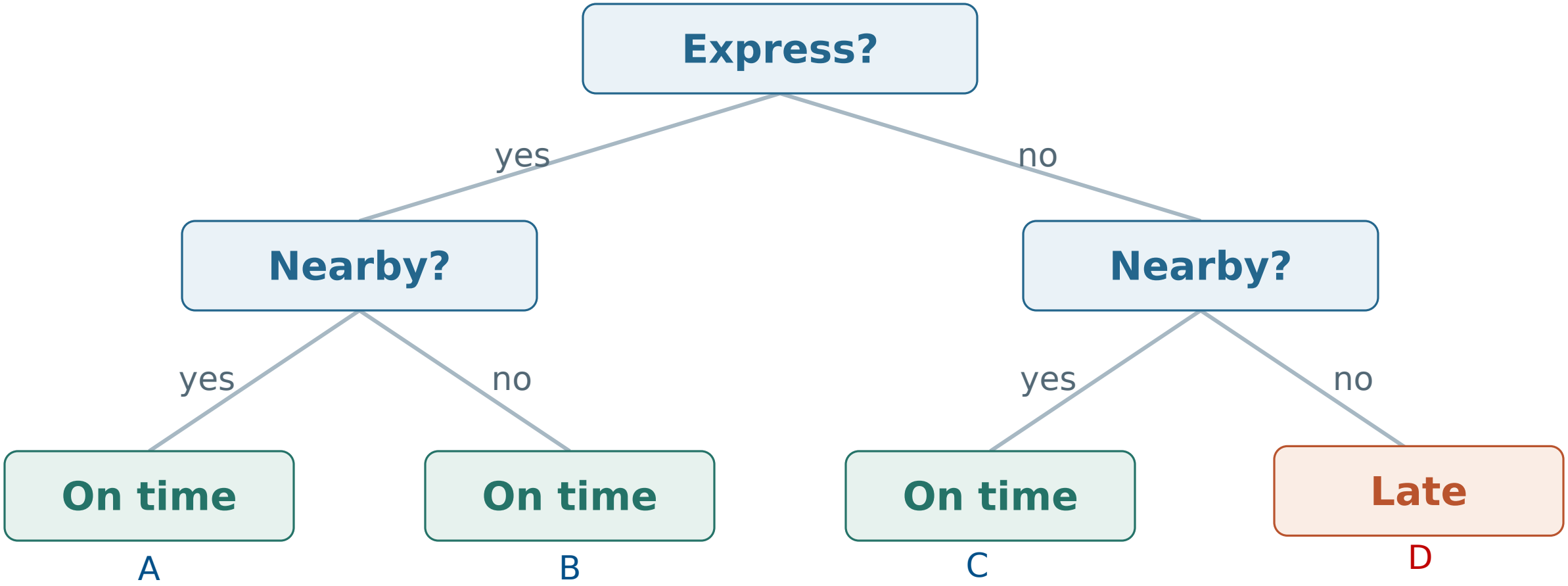
The smallest rule misses the distinction needed for D.

A small tree fits the useful distinction



3 leaves, 2 questions, 4 of 4 training predictions correct.

An unnecessary question



4 leaves. Same predictions as the 3-leaf tree.

Accuracy and tree size

Predictive aim	Accurate predictions on relevant new samples.
Complexity preference	Prefer fewer questions when predictive performance is comparable.
Why size matters	A smaller tree is easier to inspect and can reduce sensitivity to training detail.

Greedy recursive construction

Search problem	Many complete trees are possible. A root choice shapes every later group.
Greedy step	Choose the most promising question at each step.
Recursion	Repeat within each child. Stop according to a growth rule.

Purity guides the next question

Class mixture	A pure group has one class label. A mixed group may need another distinction.
Local heuristic	Choose the question with the largest size-weighted reduction in class mixture.
Connection to the goal	Cleaner groups can need fewer further questions.

Impurity guides growth. It does not assess complete-tree size or future accuracy.

A general score for a candidate question

A question q partitions node A into children $A_1, \ldots, A_k$.

$$\Delta I(q;\, A) \;=\; I(A) \;-\; \Sigma_j \left[n_j \,/\, n\right] I(A_j)$$

n = samples in A. n_j = samples in child A_j.

Choose the admissible question with the largest reduction.

Task	Average node criterion I
Classification	Gini impurity or entropy
Regression, later	Mean squared error around the leaf mean

The recursive tree-learning algorithm

Start at the root with all training samples. Repeat this routine within each child.

1. Fit a provisional leaf prediction from the samples at this node.
2. If the node is pure, has no valid split, or reaches a growth limit, return the leaf.
3. Partition this node’s samples for every valid q. Compute each gain ΔI.
4. Choose q* with the largest gain. Return the leaf if the minimum-gain rule rejects it.
5. Create a question node. For each child, run this same algorithm on its samples.

Classification leaf: majority class. Validation selects growth limits or pruning.

Gini impurity

Let $p_1, \ldots, p_m$ be the class proportions in node A.

$$G(A) = 1 - \sum {p_i}^2$$

Binary case:

Pure group: $p_1 = 0, p_2 = 1 \rightarrow G = 0$

Equal mixture: $p_1 = p_2 = \frac{1}{2} \rightarrow G = \frac{1}{2}$

$$\text{Gini Reduction} = G(\text{parent}) - E(G(\text{child}))$$

Entropy : another impurity measure

Let $p_1, \ldots, p_m$ be the class proportions in node A.

$$H(A) = -\sum p_j \log_2 p_j$$

Binary case:

Pure group: $p_1 = 0, p_2 = 1 \rightarrow H = 0$

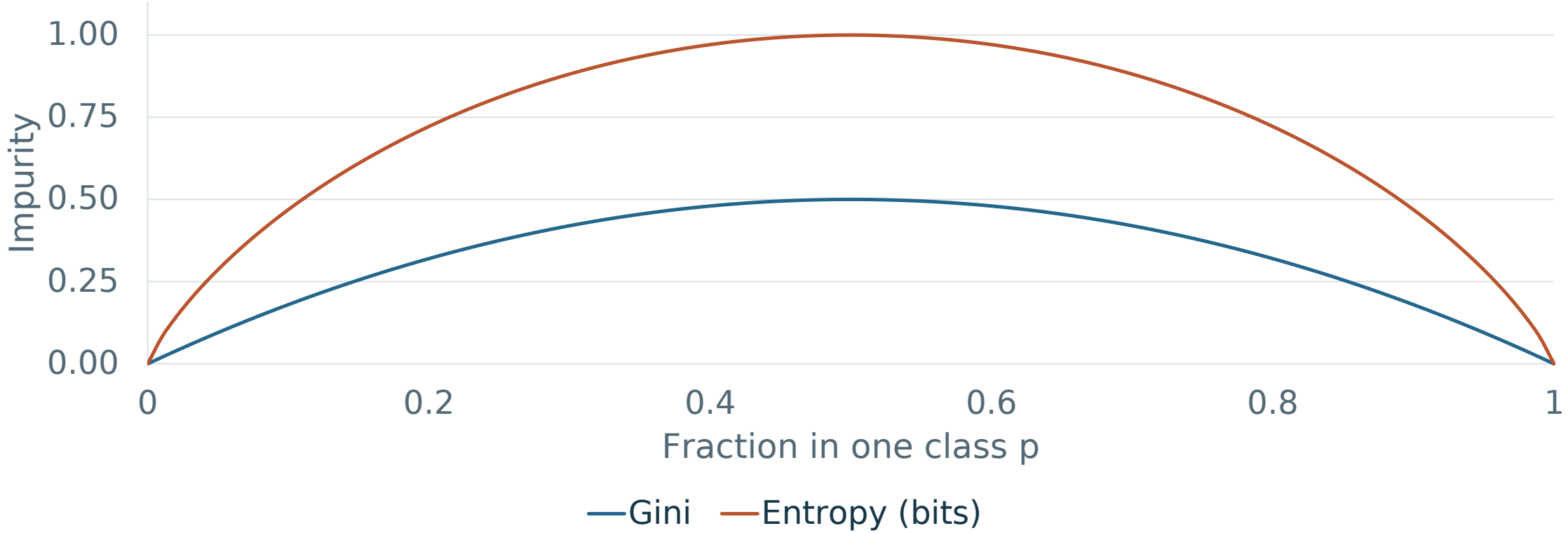
Equal mixture: $p_1 = p_2 = \frac{1}{2} \rightarrow H = 1$

Set $0 \log_2 0 = 0$.

$$\text{Information gain} = H(parent) - E(H(child)).$$

Gini and entropy as impurity measures

Binary class case



Both reward purer groups. Their scales and exact split rankings can differ.

Two features scored with Gini and entropy

ID	A	B	Class
1	yes	yes	+
2	yes	yes	+
3	yes	no	+
4	yes	no	−
5	no	no	+
6	no	no	−
7	no	no	−
8	no	no	−

Both select B.

Parent: (4+, 4−), G = .500, H = 1 bit

A: yes (3+, 1−), no (1+, 3−)

$$= .125 \Delta G = .500 - [(4/8)(.375) + (4/8)(.375)]$$

$$= .1887\ bits \Delta H = 1 - [(4/8)(.8113) + (4/8)(.8113)]$$

B: yes (2+, 0−), no (2+, 4−)

$$\approx .1667 \Delta G = .500 - [(2/8)(0) + (6/8)(.4444)]$$

$$\approx .3113\ bits \Delta H = 1 - [(2/8)(0) + (6/8)(.9183)]$$

The email-classification problem

Inputs	Thread membership, link presence, known sender, urgent language, attachment.
Training target	Spam / not spam for each of 16 labeled emails.
Split criterion	Gini impurity reduction, computed within the node being fitted.
Fitting task	Choose feature questions and leaf predictions, then control tree size.

Email training samples 1

ID	Known Sender	Contains Link	Urgent	Existing Thread	Has Attachments	Spam
1	1	1	0	1	0	0
2	0	1	0	1	1	0
3	1	0	0	1	0	0
4	0	0	1	1	0	0
5	1	1	1	1	1	0
6	1	0	1	1	1	0
7	0	0	1	1	0	1
8	0	1	1	0	0	1

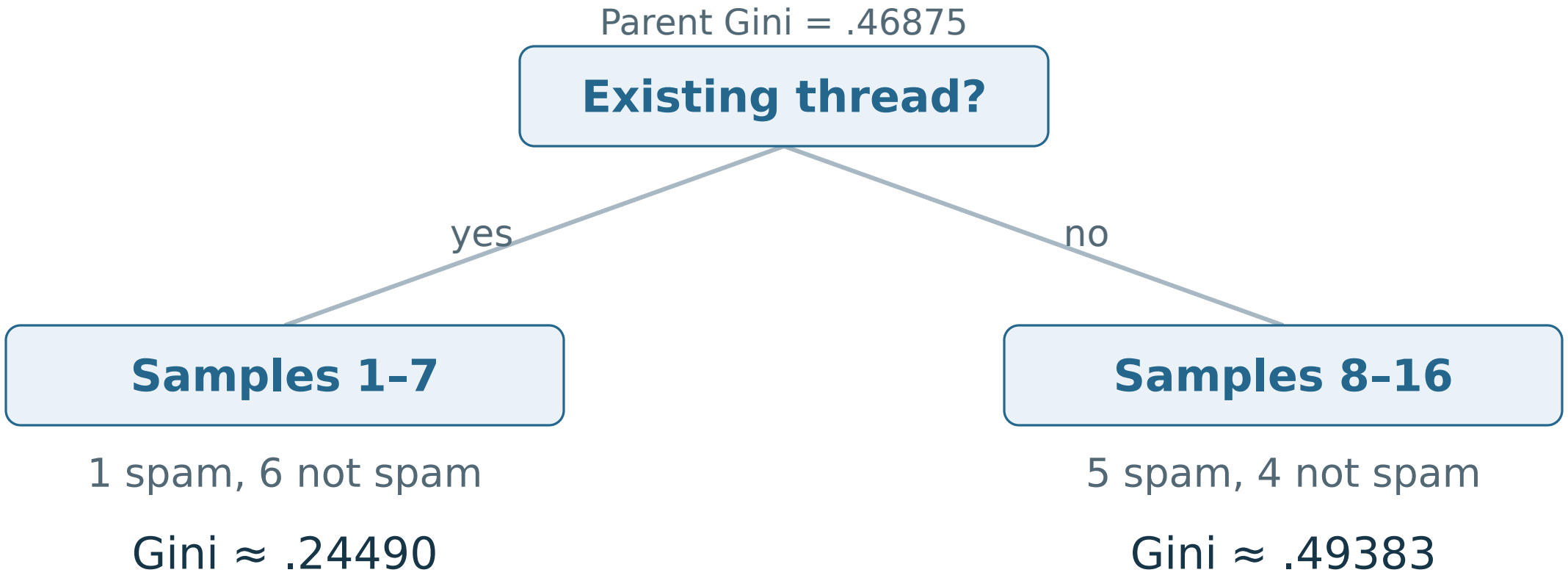
1 = yes, 0 = no.

Email training samples 2

ID	Known Sender	Contains Link	Urgent	Existing Thread	Has Attachments	Spam
9	1	1	0	0	0	1
10	0	1	0	0	1	1
11	1	1	1	0	1	1
12	1	1	0	0	1	0
13	0	0	1	0	0	0
14	1	0	0	0	0	0
15	0	0	0	0	1	0
16	1	0	1	0	1	1

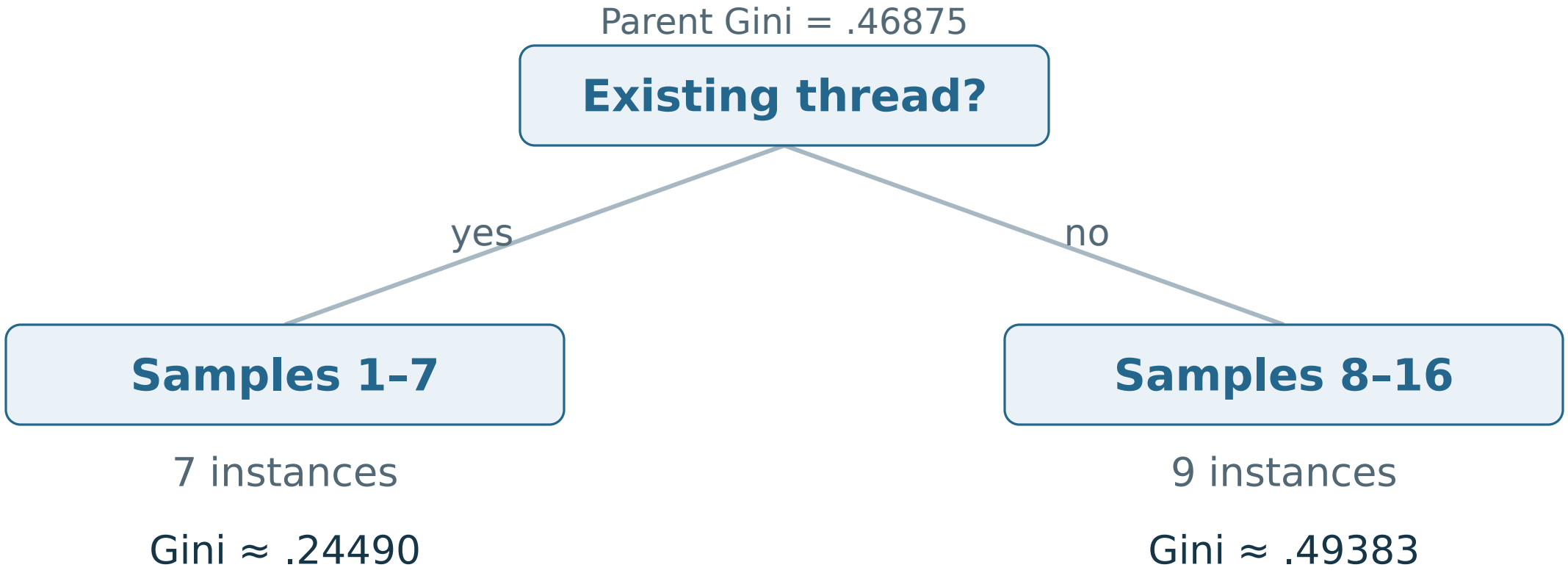
1 = yes, 0 = no.

The two child impurities



Each child uses its own class proportions.

A size-weighted split gain



Weighted child Gini = (7/16)(.24490) + (9/16)(.49383) ≈ .38492

Gain = .46875 − .38492 ≈ .08383

The first split choice

Candidate root question	Gini reduction
Existing thread	**.08383**
Contains link	.03125
Sender known	.00446
Urgent language	.03125
Attachment	.00000

Choose the largest local Gini reduction among the available questions.

Choosing the no-thread question

Thread = no

Samples 8–16

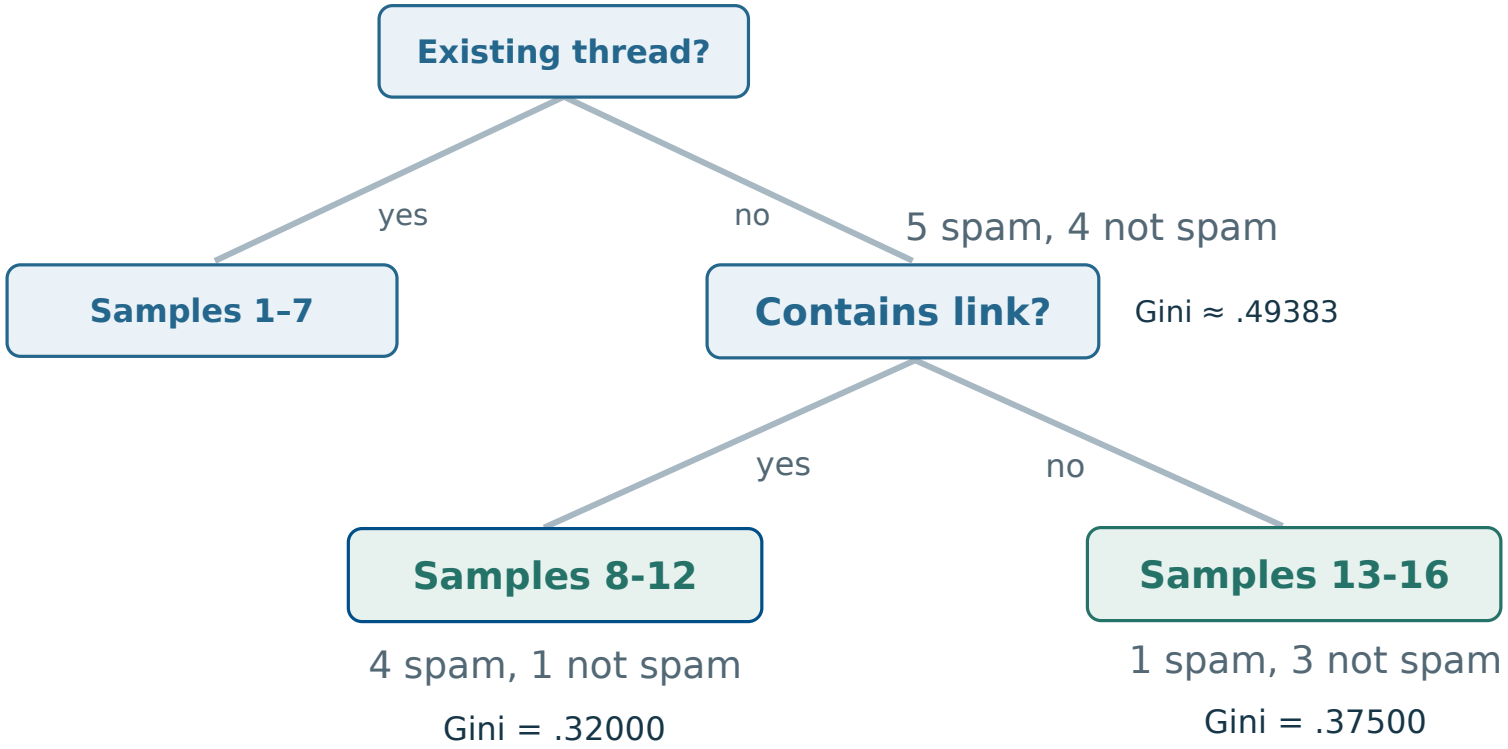
5 spam, 4 not spam

Gini = 40/81 ≈ .49383

Candidate question	Gini reduction
Contains link	**.14938**
Urgent language	.06049
Sender known	.00494
Attachment	.00494
Existing thread	Not a split

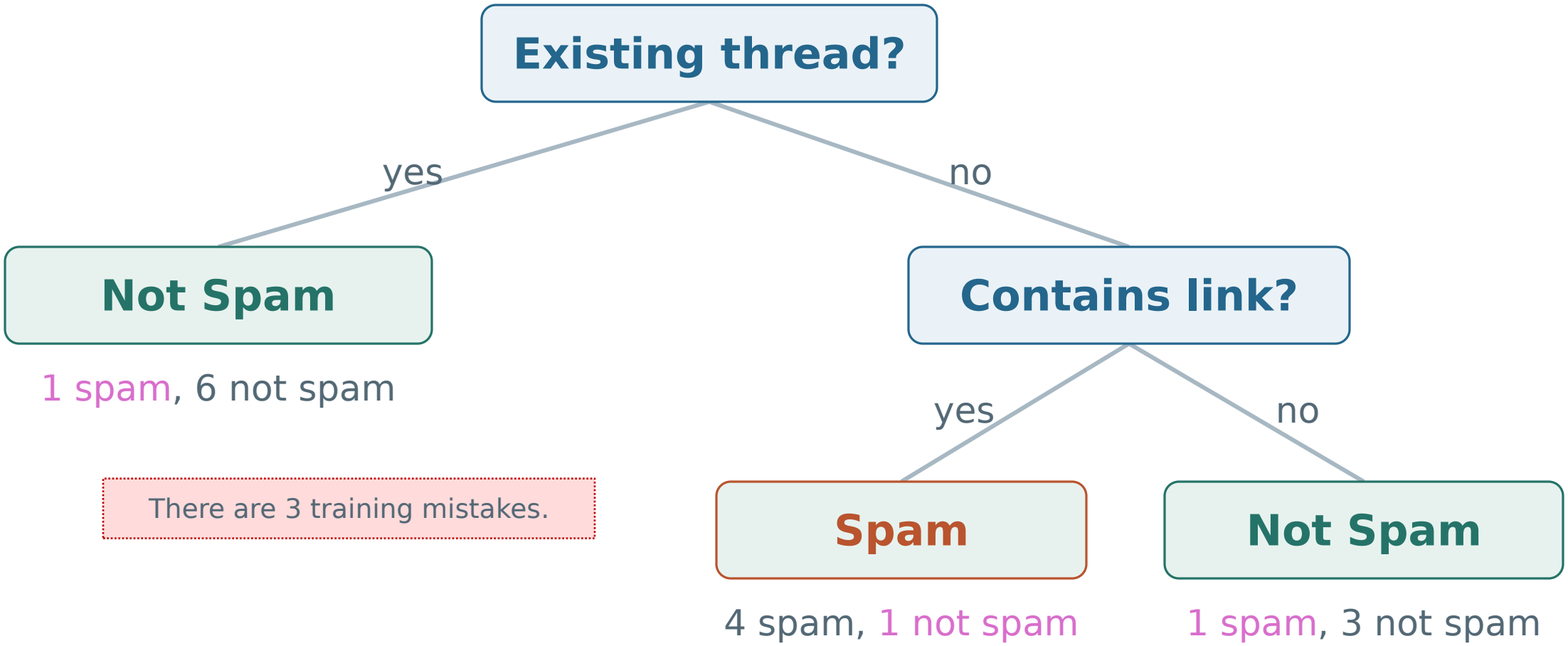
All gains use only this node’s samples. Existing thread is constant here.

Fitting the no-thread child

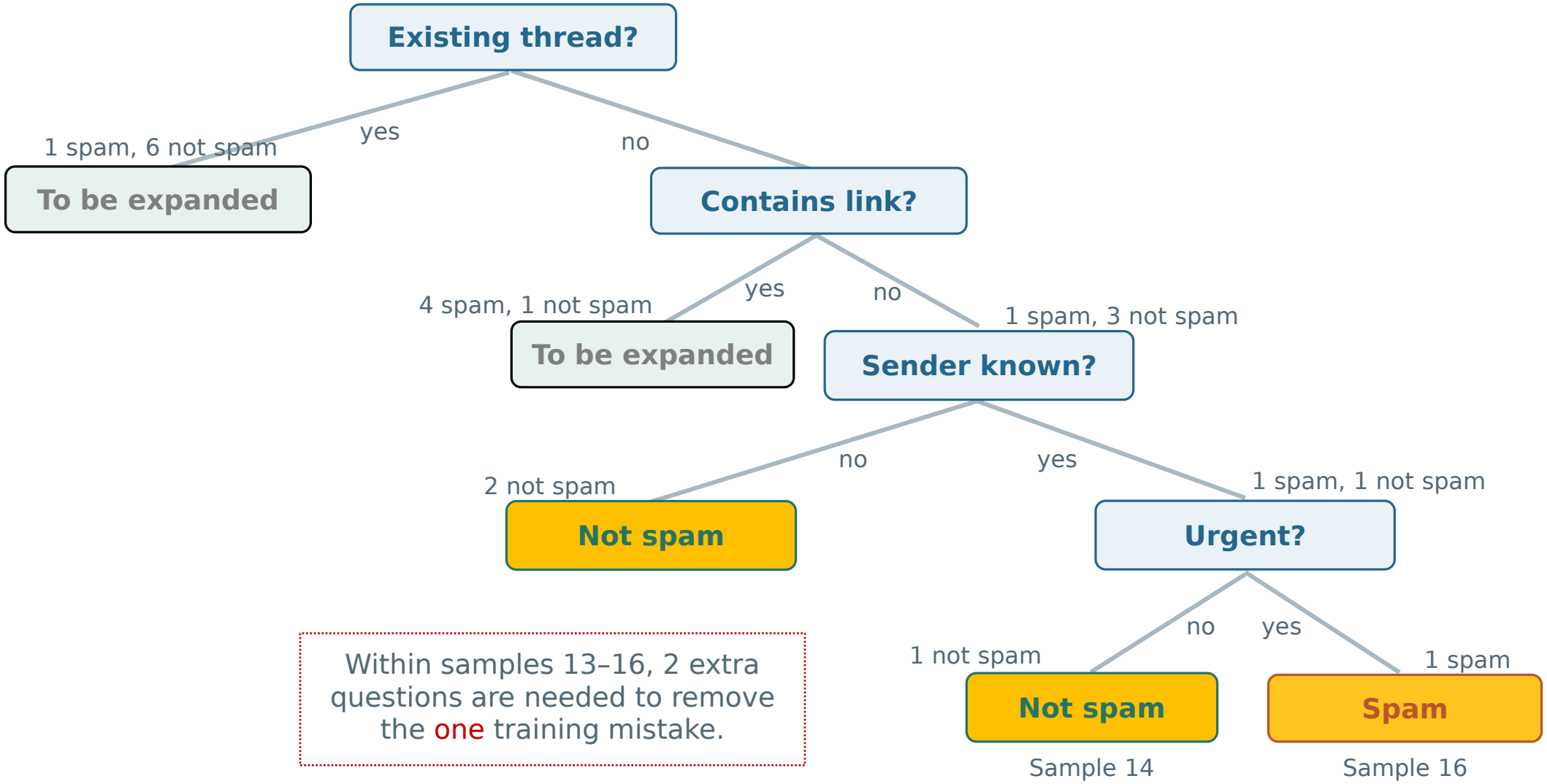


Local gain = .49383 − [(5/9)(.32000)+(4/9)(.37500)] ≈ .14938.

We can stop here

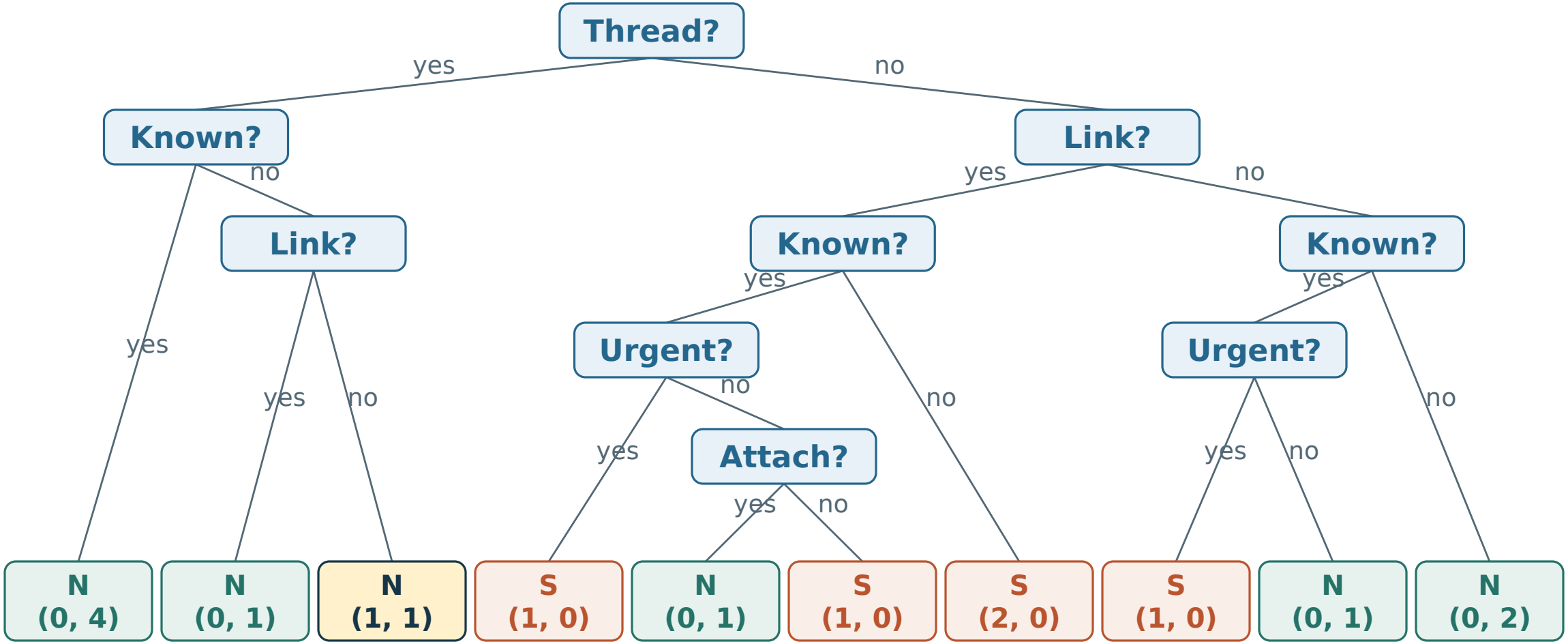


Further growth fits training exceptions



The fully expanded email tree

10 leaves. One mixed leaf: emails 4 and 7 have identical inputs and opposite labels.

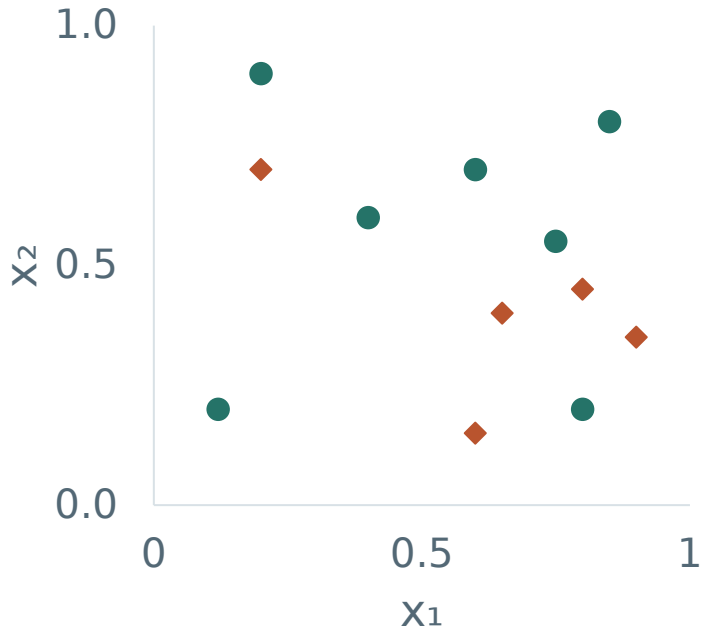


S = spam. N = not spam. Counts: (spam, not spam). Yellow leaf: emails 4 and 7.

Model flexibility on the same training data

Schematic example. Green circles: class 0. Orange diamonds: class 1. Blue lines: boundaries.

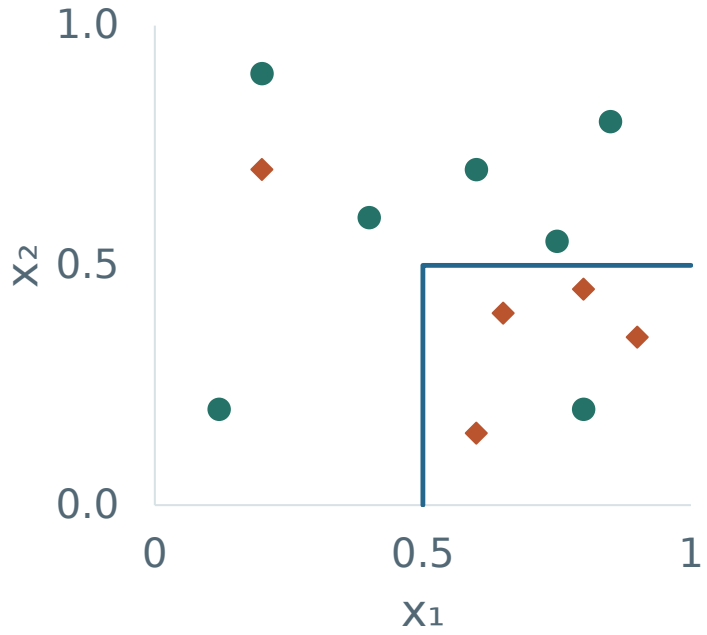
Too small



Predict 0 everywhere

Training errors: 5/12

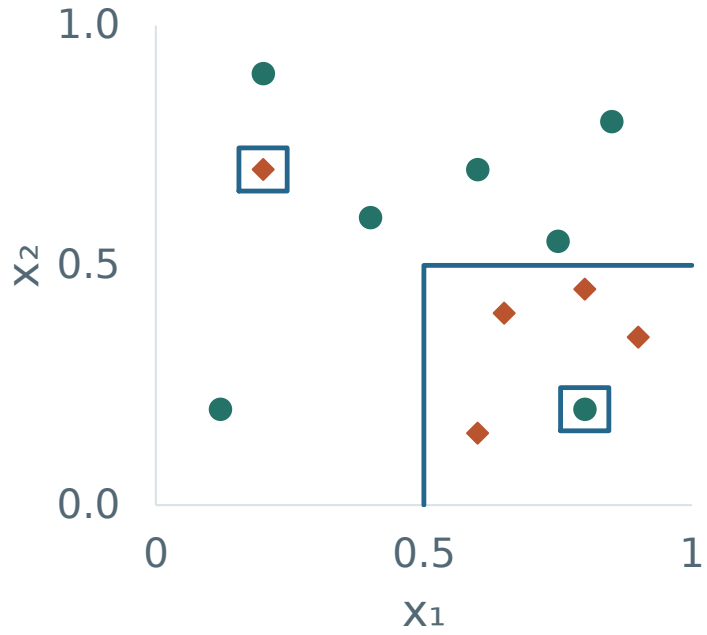
Useful structure



Class 1: lower right

Training errors: 2/12

Fits exceptions



Extra prediction regions

Training errors: 0/12

Underfitting and overfitting

Underfitting	The fitted model misses useful predictive structure, even in training.
Overfitting	Fitting extra training detail reduces training error but harms prediction on new samples.
Evidence	Inspect training and validation errors.

Validation predictions for three new emails

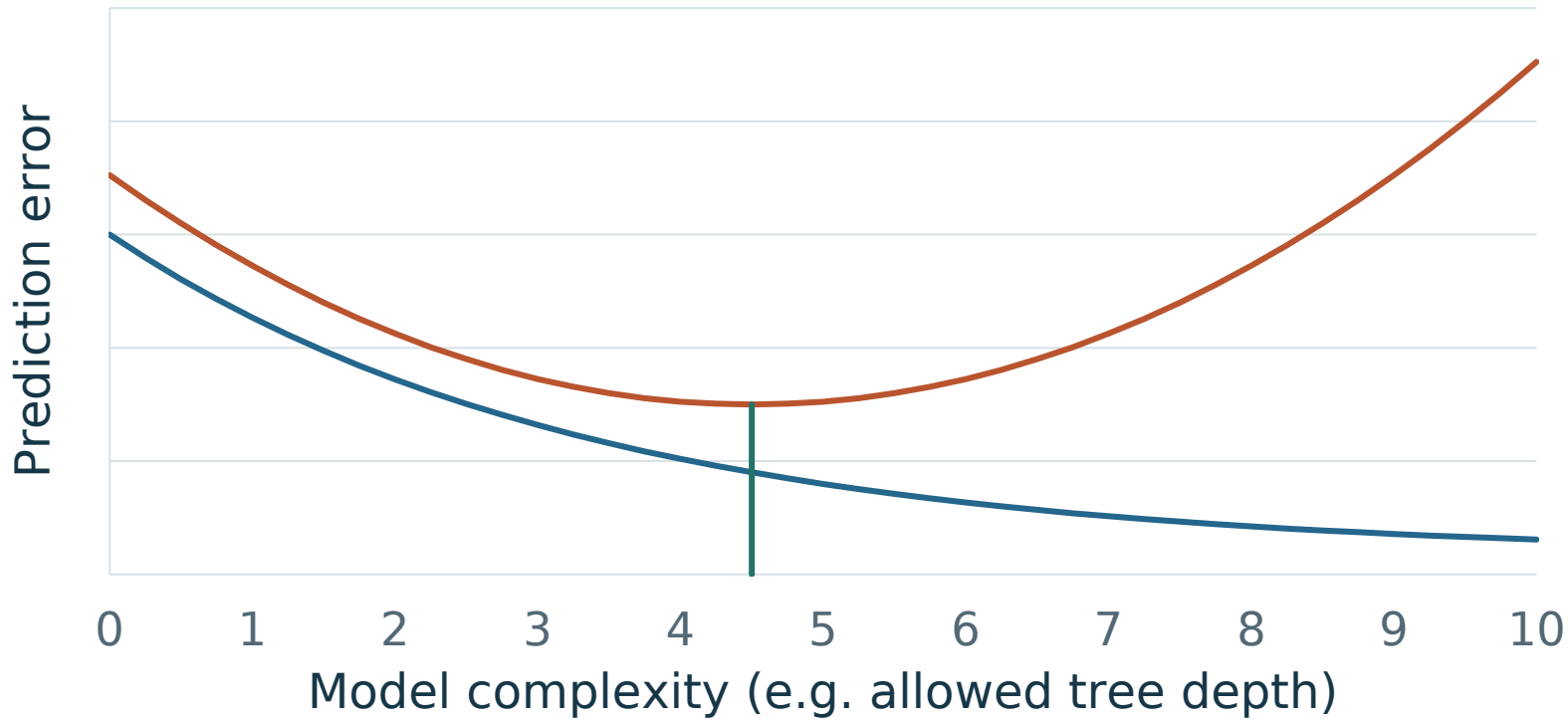
Email	Thread	Link	Known	Urgent	Attach	Observed label
A	no	yes	no	no	no	spam
B	no	yes	yes	no	yes	spam
C	no	no	yes	yes	no	not spam

Email	1-leaf tree	Compact 3-leaf tree	Expanded 10-leaf tree
A	not spam ✗	spam ✓	spam ✓
B	not spam ✗	spam ✓	not spam ✗
C	not spam ✓	not spam ✓	spam ✗

Validation errors: 2/3, 0/3, and 2/3. Targets were held out during fitting.

Model complexity and generalization

Compare errors as model flexibility increases, using the same training and validation sets.



Schematic curves. Actual validation curves may be irregular.

Underfitting

Both errors are high.

Useful complexity

Validation error is lowest.

Overfitting

Training error falls while validation error rises.

Selecting maximum depth

Fit each setting on training data. Compare on the same validation data.

Depth limit	Training errors	Validation errors	Decision
0	6/16	2/3	Too restricted
1	5/16	1/3	Validation improves
2	**3/16**	**0/3**	**Select this depth**
3	3/16	0/3	Same score, deeper
4	2/16	2/3	Validation worsens
5	1/16	2/3	Validation worsens

Depths 2 and 3 tie. Prefer depth 2 for simplicity.

Use an untouched test set for the final assessment.

Bias: error in the average prediction

Fix x and the learning procedure. Each D is a new training set. E_D averages over these sets.

$f_D(x)$: prediction after fitting D

$f^*(x)$: true spam probability

Average prediction

$$\bar{f}(x) = E_D[f_D(x)]$$

$$\text{Bias}(x) = \bar{f}(x) - f^*(x)$$

Same email x, five fitted spam probabilities



Average = .30 **True target = .60**

Bias = .30 − .60 = −.30. The model systematically underestimates this probability.

Variance: sensitivity to the training set

$\text{Variance}(x) = E_D[(f_D(x) - \bar{f}(x))^2]$

How far do fitted predictions move around their own average?

Low variance



High variance



Variance = .0002

Variance = .08

Both averages equal .60, the true target. Their bias is the same, but their spread differs.

Controlling growth before a split

A growth rule rejects a split before adding the question.

Maximum depth 2

At most two questions on any prediction path.

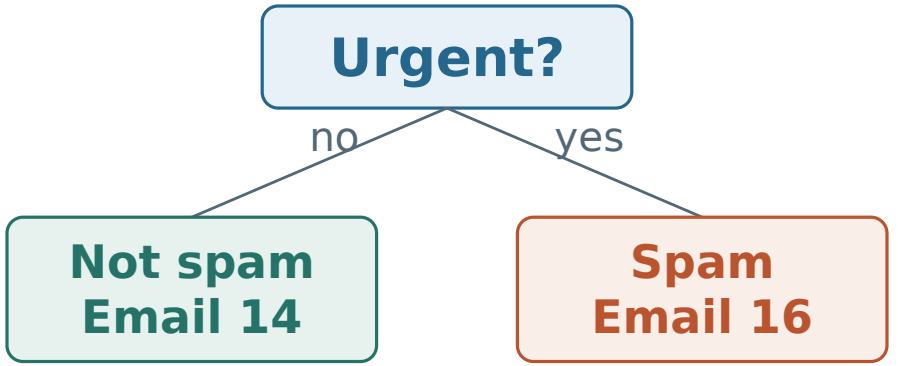
Minimum leaf size 2

Every proposed child needs at least two training samples.

Minimum gain .06

The local Gini reduction must exceed .06.

Candidate split: emails 14 and 16



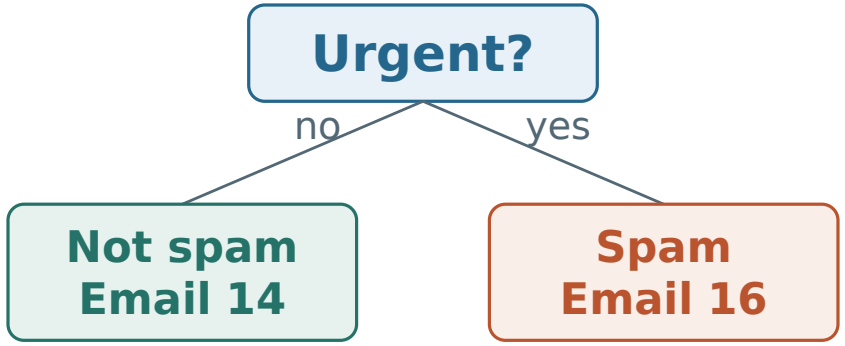
Each child has only 1 sample.

Minimum leaf size 2 rejects it.

Pruning: replacing a subtree with a leaf

Region: thread = no, link = no, sender known = yes

Keep the urgent question



Replace it with a leaf

Not spam

Training tie: predict not spam.

Validation C: urgent=yes.
Observed label: not spam.

Model in this region	Leaves	Training errors	Validation error C
Keep the question	2	0/2	1/1
Prune to a leaf	**1**	**1/2**	**0/1**

Questions for discussion

1 Why can a locally promising question lead to a poor complete tree?

2 When can a mixed leaf be a useful final prediction?

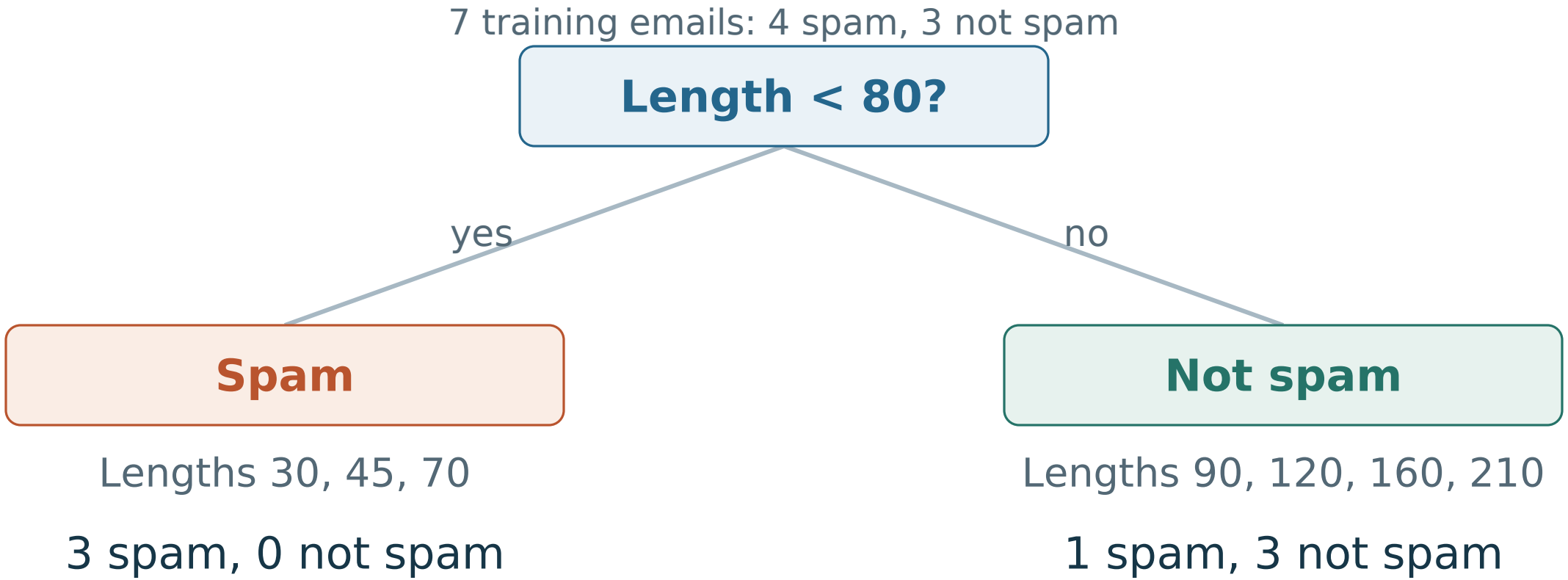
3 Which data choose tree complexity, and which assess the selected procedure?

Numerical features require a threshold

Message length	30	45	70	90	120	160	210
Spam?	yes	yes	yes	no	yes	no	no

Candidate questions:
length<37.5?, length<57.5?, length<80?, length<105?, length<140? and length<185?

A numerical candidate split



Gain at 80 = 24/49 − [(3/7)(0)+(4/7)(3/8)] = 27/98 ≈ .27551.

Comparing threshold candidates

Threshold	37.5	57.5	80	105	140	185
Gini reduction	.061	.147	.276	.085	.261	.109

Threshold selection uses the samples reaching this node.

Imputation across model families

Preprocessing can supply complete inputs to kNN, linear models, neural networks, and trees.

Numerical feature

Observed values: 10, 12, 20
Stored median: 12

New input: ? becomes 12

Categorical feature

Training values: 3 yes and 1 no
Stored mode: yes

New input: ? becomes yes

Fit replacements on training data only. Reuse them during validation and prediction.

Encoding that a value is missing

These feature encodings can be used across model families.

Categorical input

Categories: yes, no, missing

Original	One-hot vector
yes	(1, 0, 0)
no	(0, 1, 0)
?	(0, 0, 1)

Numerical input plus an indicator

Original	Filled value	Missing?
12	12	0
?	12	1

An imputed 12 differs from an observed 12.

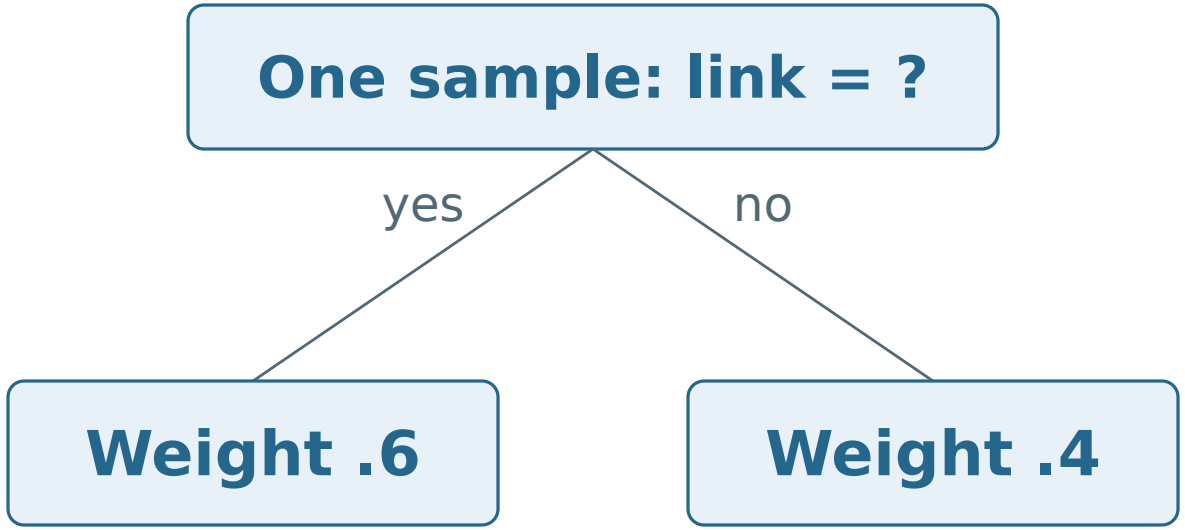
The extra feature distinguishes imputed values from observed values.

Fractional routing at a tree node

Estimate branch proportions from known feature values at this node.

Toy training data

Link	Spam	Not spam	Total
yes	4	2	6
no	1	3	4
?	2	0	2



r_{yes} = 6/10 = .6 **r_{no} = 4/10 = .4**

A sample of weight w contributes .6w and .4w. Its total weight remains w.

Learning with fractional samples

2 new instances with missing link values belonging to spam class exist
Each missing-link spam sample contributes .6 to yes and .4 to no child.

Child	#spams	#Not-spam	#Instances	Spam portion	Not-Spam Proportion	Gini
yes	4 + 2(.6) = 5.2	2	7.2	5.2/7.2 ≈ .722	2/7.2 ≈ .278	.401
no	1 + 2(.4) = 1.8	3	4.8	1.8/4.8 = .375	3/4.8 = .625	.468

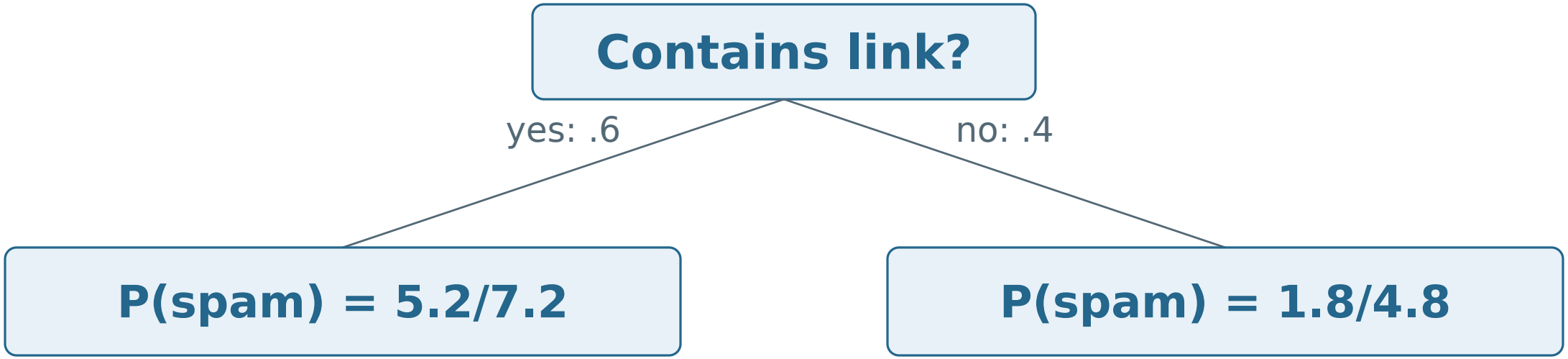
Class portion = class weight / total weight

Gini gain = .4861 − [(7.2/12)(.401) + (4.8/12)(.468)]

≈ .0579

Prediction with fractional routing

New email x: link = ?. Follow both branches with their stored proportions.



P(spam | x) = .6(5.2/7.2) + .4(1.8/4.8) = 7/12 ≈ .583

P(not spam | x) = 5/12 ≈ .417. Predict spam.

If another tested feature is missing, split again and multiply the path fractions.

A regression leaf

A numerical target requires a numerical leaf prediction c.

Choose c to minimize $\sum (y_i - c)^2$ over samples in the leaf.

With n leaf samples, the best constant is $\bar{y} = (1/n)\sum y_i$.

For (10,12,14,20), mean = 14. The sum of squared errors (SSE) is 56.

Regression impurity and expected child error

Parent size: $n = n_L + n_R$. Fit a separate mean in each child.

$$\mathbf{MSE(A) = (1/n_A) \sum_{i \in A} (y_i - \bar{y}_A)^2}$$

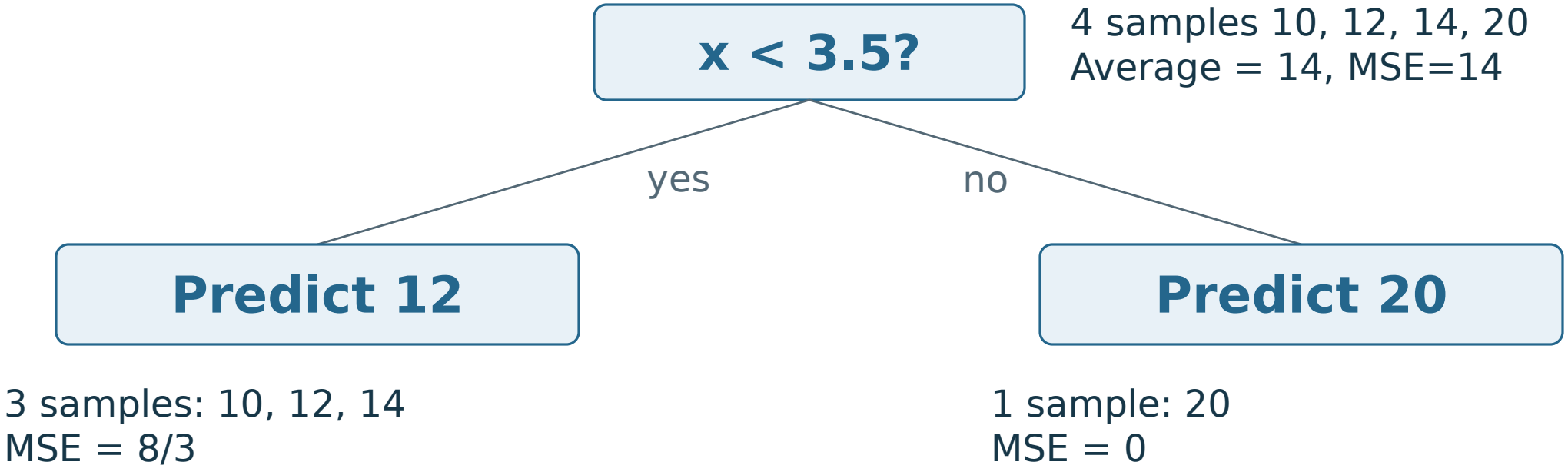
Expected child error

$$\mathbf{E_{child} = (n_L/n)\, MSE(L) + (n_R/n)\, MSE(R)}$$

Improvement = MSE(parent) − E_{child}

A regression split: expectation and total error

x = 1, 2, 3, 4 y = 10, 12, 14, 20



Child expectation = (3/4)(8/3) + (1/4)(0) = (8+0)/4 = 2

Improvement = 14 − 2 = 48/4 = 12.

Comparing regression thresholds

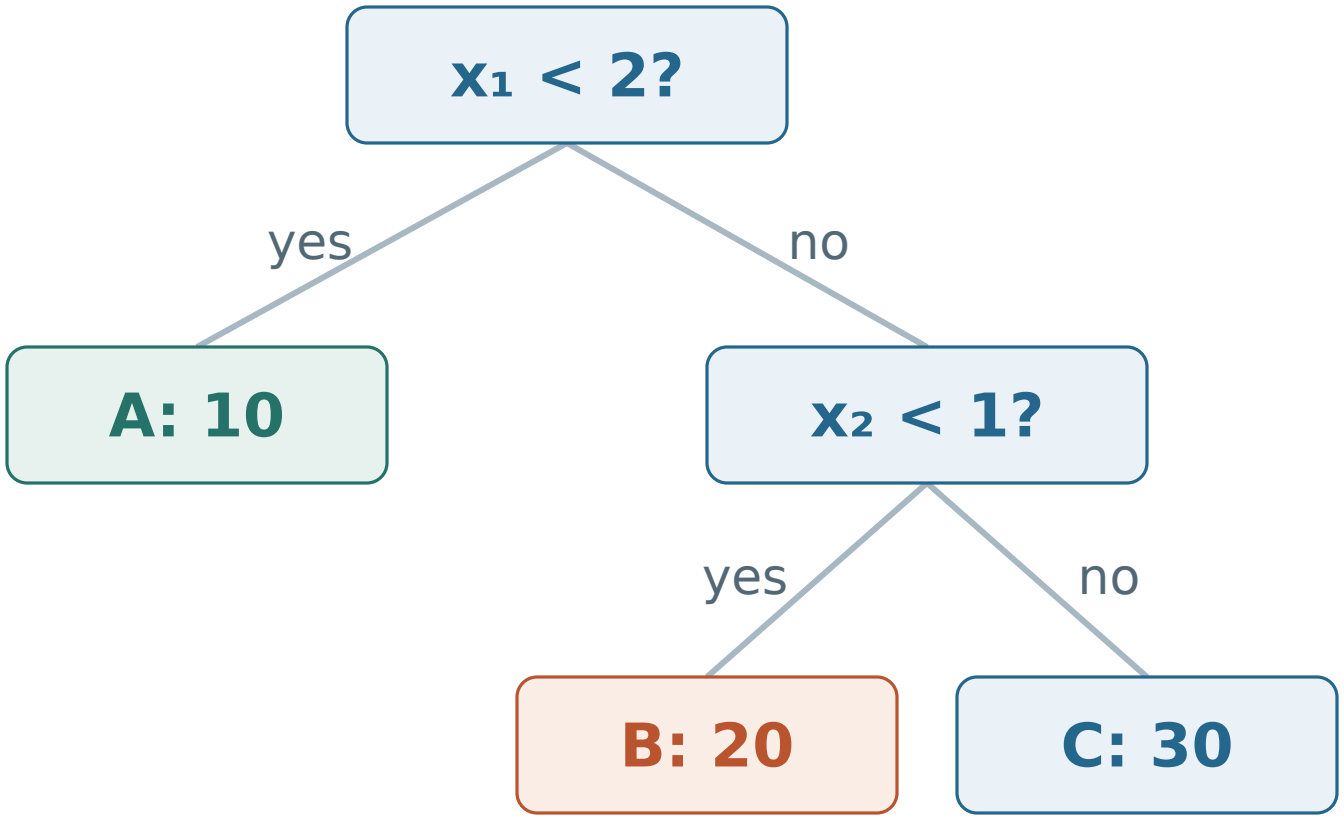
Same four training samples. Parent MSE = 56/4 = 14.

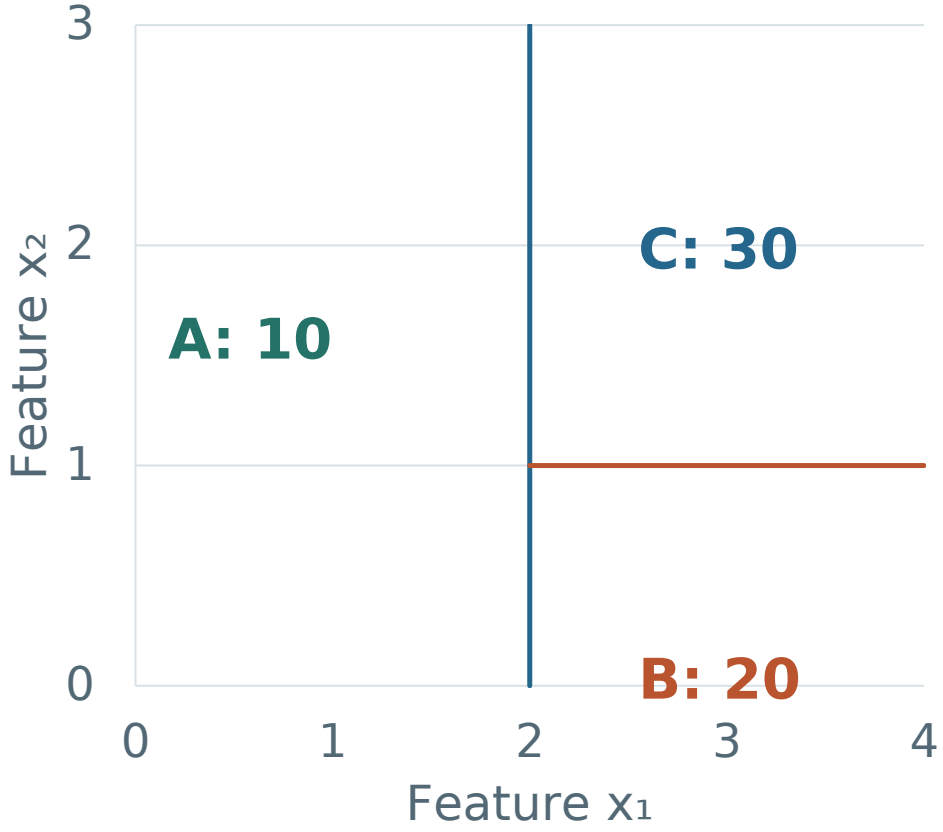
Threshold x < t	Child SSE sum	Expected child MSE	MSE reduction
1.5	104/3 ≈ 34.667	26/3 ≈ 8.667	16/3 ≈ 5.333
2.5	20	5	9
3.5	**8**	**2**	**12**

Expected child MSE = summed child SSE / 4.

Both criteria select 3.5. The difference is a constant scale factor.

A tree and its two-feature partition





The x_2 question divides only the region $x_1 \geq 2$.

Read yourself: information gain ratio

Many tiny branches can inflate information gain
Gain ratio adjusts for fragmentation

$$\text{Child fractions } q_j = \frac{n_j}{n}$$

$$\text{Split information} = -\sum_j q_j \log_2 q_j$$

$$\text{Gain ratio} = \frac{\text{information gain}}{\text{split information}}$$

.

Bibliographical notes

Breiman et al. (1984)

Classification and Regression Trees.

Quinlan (1986)

Induction of decision trees.

Quinlan (1993)

C4.5: Programs for Machine Learning.